Arquitectura del sistema de Prosta-AI

Yeison Bolaños Ruiz

Juan David Riascos Erazo

Sergio Gabriel Meneses Silva

Yimis Buenaventura

Fernando José Angulo Núñez

Facultad de Ingeniería, Universidad Santiago de Cali

Programa de Ingeniería de Sistemas

Gestión de Proyectos de TI

Jair Enrique Sanclemente Castro

22 de octubre de 2026

Arquitectura del sistema de Prosta-AI

Definición

Sommerville (2011) define el modelo de arquitectura de un sistema como una descripción corta y manejable de cómo se organiza el sistema y de cómo interoperan sus componentes. A diferencia de la arquitectura de software, que detalla los módulos internos del backend, esta vista se queda en el nivel de los bloques grandes: qué partes tiene Prosta-AI, qué información intercambian y dónde se ejecuta cada una.

Prosta-AI es una aplicación web académica para cargar estudios de resonancia magnética de próstata desidentificados, segmentarlos en 3D y visualizar la región de interés (ROI). El sistema no ofrece diagnóstico ni recomendaciones clínicas. Este documento corresponde al paquete 1.3.2.3 de la EDT (arquitectura del sistema) y a la actividad 1306 del Ciclo 2; el detalle interno de los componentes está en la arquitectura de software del proyecto (Equipo Prosta-AI, 2026b).

Tipos de arquitectura y elección

Sommerville (2011) señala que la arquitectura debe elegirse según los requisitos no funcionales, porque de ella dependen el rendimiento, la protección, la disponibilidad y la mantenibilidad del sistema. Por eso, antes de comparar las alternativas, se identifican los atributos que más pesan en Prosta-AI:

1. **Rendimiento:** cada estudio debe procesarse en dos minutos o menos (requisito 5.1), con una inferencia 3D exigente y equipos de cómputo limitados (riesgo 1 del Acta; Equipo Prosta-AI, 2026a).
2. **Protección de datos:** no puede almacenarse ningún dato que identifique a una persona (requisito 2.2 y criterio vital del Acta).
3. **Mantenibilidad y reproducibilidad:** el modelo debe poder cambiarse sin rehacer la aplicación, con la configuración versionada (requisitos 3.4 y 5.4).

4. **Restricciones del proyecto:** cinco estudiantes con dedicación parcial, un stack ya definido (React, FastAPI, PostgreSQL y PyTorch/MONAI), sin presupuesto y con entrega final el 09/12/2026.
5. **Disponibilidad:** al ser un prototipo académico sin despliegue en producción, basta con que funcione durante las pruebas y la demostración.

Con esos atributos se compararon tres tipos de arquitectura (Tabla 1).

Tabla 1
Comparación de tipos de arquitectura para Prosta-AI

Criterio	**Monolítica**	**Cliente-servidor en capas**	**Microservicios**
Organización	Una sola aplicación que integra interfaz, lógica y datos.	Un cliente web separado de un servidor organizado en capas (presentación, servicios, procesamiento y datos).	Varios servicios pequeños e independientes que se comunican por red.
Rendimiento (5.1)	Bueno para un usuario; la inferencia compite con la interfaz.	Bueno; la interfaz sigue respondiendo mientras el servidor procesa.	Permite escalar la inferencia, pero agrega latencia de red.
Protección de datos (2.2)	Un solo punto de control.	Los datos quedan solo en el servidor; el cliente nunca los almacena.	Los datos viajan entre servicios; hay más puntos que proteger.
Mantenibilidad (3.4, 5.4)	Baja: un cambio en el modelo afecta toda la aplicación.	Alta: cada capa cambia sin afectar a las demás.	Alta, pero cada servicio exige su propio despliegue y versionado.
Ajuste al stack y al equipo	No aprovecha la separación natural entre React y FastAPI.	Coincide con el stack definido y con los roles del equipo.	Exige experiencia en contenedores y orquestación que el equipo no tiene.
Costo y tiempo	Bajo.	Bajo; un solo servidor y servicios gratuitos.	Alto en configuración, infraestructura y pruebas.
Decisión	**Descartada.**	**Elegida.**	**Descartada.**

Se elige la **arquitectura cliente-servidor en capas**. Es la que mejor responde a los atributos del proyecto: separa la interfaz del procesamiento, de modo que el navegador sigue respondiendo mientras el servidor ejecuta la inferencia; concentra los estudios en el servidor, lo que facilita cumplir la protección de datos; y permite cambiar el modelo de segmentación sin

tocar la interfaz ni la base de datos. Además, coincide con el stack que fija el Acta: React en el cliente y FastAPI, PyTorch/MONAI y PostgreSQL en el servidor.

La arquitectura monolítica se descarta porque mezclaría la interfaz con la inferencia y haría que cualquier cambio en el modelo afectara a toda la aplicación. Los microservicios se descartan por su complejidad: exigirían contenedores, comunicación entre servicios y despliegues independientes, algo desproporcionado para cinco estudiantes con un alcance académico y sin presupuesto.

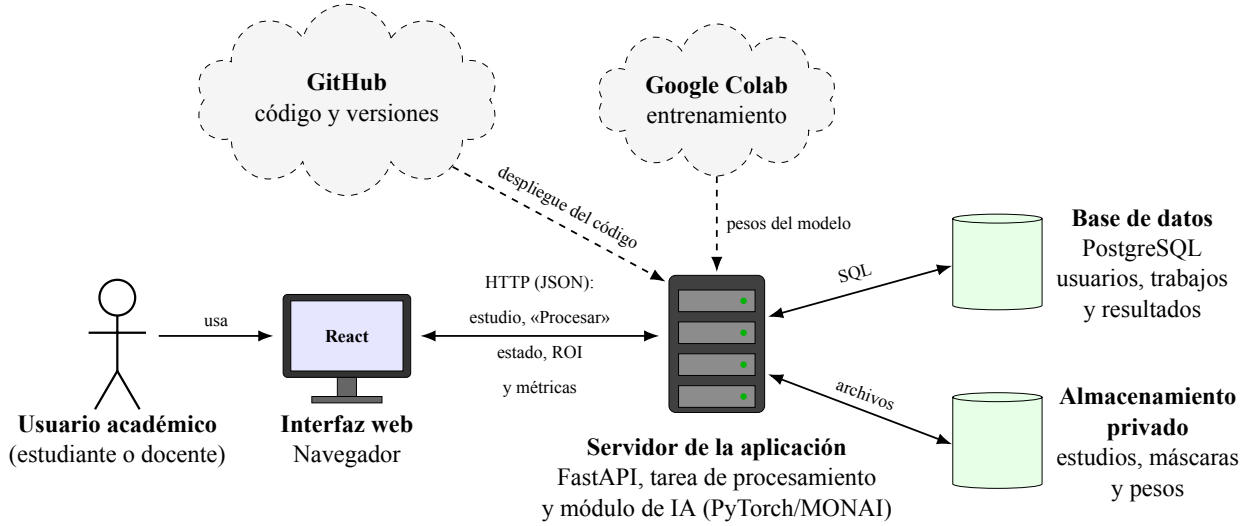
La elección toma una ventaja de la arquitectura monolítica: como mínimo viable, todo el lado del servidor se ejecuta en un único proceso, con sus módulos bien separados. Si las mediciones del riesgo 1 muestran que la inferencia bloquea la API, el procesamiento puede pasar a un proceso aparte sin cambiar el resto del sistema (Equipo Prosta-AI, 2026b).

Diagrama de arquitectura del sistema

La Figura 1 presenta una visión abstracta del sistema: quién lo usa, desde dónde accede y con qué componentes se relaciona para funcionar. Prosta-AI tiene un solo tipo de usuario, el usuario académico, porque el Acta excluye los roles y la administración de cuentas. El usuario trabaja desde un navegador web, que se comunica por HTTP con el servidor de la aplicación; el servidor guarda la información en la base de datos y los archivos en un almacenamiento privado. Google Colab y GitHub son servicios externos que no intervienen en el uso normal de la aplicación.

Figura 1

Diagrama de arquitectura del sistema Prosta-AI



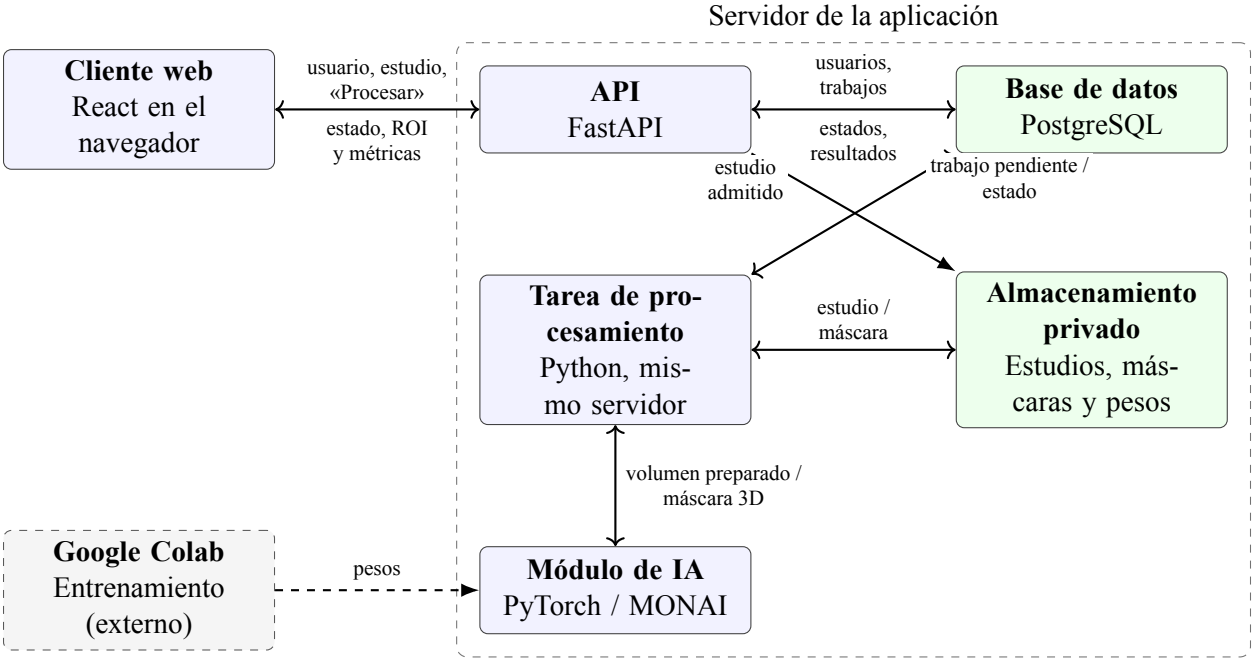
Nota. Las líneas continuas son el uso normal de la aplicación; las punteadas, servicios externos que solo se usan para entrenar el modelo y desplegar el código. Elaboración propia.

Componentes del servidor

La Figura 2 amplía el servidor de la aplicación: muestra sus bloques internos y la información que circula entre ellos. El entrenamiento del modelo ocurre fuera del sistema, en Google Colab, y solo entrega los pesos que usa el módulo de IA.

Figura 2

Componentes internos del servidor de la aplicación



Nota. Las cajas punteadas indican el servidor de la aplicación y el entorno externo de entrenamiento. Elaboración propia.

Interoperación de los componentes

El flujo principal del sistema sigue los casos de uso CU01 a CU05:

1. El usuario ingresa desde el navegador con su nombre de usuario y contraseña (CU01). La API los verifica en PostgreSQL; si el usuario no existe, se crea confirmando la contraseña.
2. El usuario carga un estudio DICOM o NIfTI (CU02). La API comprueba el formato y el tamaño y guarda el archivo en el almacenamiento privado.
3. El usuario pulsa «Procesar» (CU03). La API registra un trabajo pendiente en PostgreSQL y responde de inmediato, sin esperar la segmentación.
4. La tarea de procesamiento toma el trabajo, valida la integridad y elimina los metadatos identificables, y prepara el volumen.
5. El módulo de IA recibe el volumen preparado y devuelve la máscara 3D. La tarea genera la ROI, calcula Dice e IoU si existe una referencia y guarda la máscara y el resultado.

6. Durante todo el proceso, la tarea actualiza el estado del trabajo en PostgreSQL. El cliente consulta ese estado a través de la API y, al terminar, muestra la ROI con el aviso experimental (CU04).
7. Los estudios y resultados quedan registrados para consultarlos después (CU05).

Vista física

Como prototipo académico, el sistema se ejecuta con los recursos disponibles del equipo y sin costos. La Tabla 2 indica dónde corre cada parte y la Figura 3 resume el despliegue.

Tabla 2
Nodos físicos del sistema

Nodo	Qué ejecuta	Observación
Equipo del usuario	Navegador web con la interfaz en React.	No guarda estudios ni resultados.
Servidor de la aplicación	API, tarea de procesamiento, módulo de IA, PostgreSQL y almacenamiento privado.	Equipo de un integrante o cualquier servidor de demostración; un solo backend.
Google Colab	Entrenamiento y ajuste del modelo con GPU gratuita.	Fuera del flujo de uso; solo entrega los pesos.
GitHub	Código, configuración versionada y documentación.	Rama principal protegida y etiquetas por ciclo.

Figura 3
Vista física del despliegue



Nota. Las líneas punteadas son intercambios fuera del uso normal de la aplicación. Elaboración propia.

Referencias

Equipo Prosta-AI. (2026a). *Acta de constitución del proyecto Prosta-AI* [Documento no publicado]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026b). *Arquitectura de software de Prosta-AI* (Versión 1.1) [Documento no publicado]. Universidad Santiago de Cali.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.